

给范畴论背景读者的 AI 算法讲义：神经网络、Transformer 与 LLM

2026 年 6 月 27 日

摘要

本文面向具有范畴论或 topos 背景、但不熟悉现代深度学习算法细节的读者。目标不是介绍工程调参，而是建立一套统一符号，说明监督学习、神经网络、反向传播、卷积网络、图神经网络、Transformer 和大语言模型的基本数学对象。后续若要讨论这些对象能否被组织成范畴、预层、层、stack 或 topos，首先必须知道 AI 侧的对象、态射候选、局部结构和训练目标是什么。

1 读者契约

范畴论读者容易把神经网络想成一个态射

$$f : X \rightarrow Y.$$

这个想法有用，但还不够。机器学习中的一个模型通常包含至少四层数据：

数据空间 参数空间 损失函数 训练算法.

因此更准确的最小记号是

$$N_\theta : \mathcal{X} \rightarrow \mathcal{Y}, \quad \theta \in \Theta,$$

其中 Θ 是参数空间。训练并不是只研究某个固定态射 N_θ ，而是在 Θ 中寻找使损失较小的参数。

原则 1.1 (先分清三种对象). 讨论 AI 与范畴论的关系时，应区分：

1. 模型族： $\{N_\theta : \mathcal{X} \rightarrow \mathcal{Y}\}_{\theta \in \Theta}$ ；
2. 训练过程： $\theta_0, \theta_1, \dots, \theta_T$ ；
3. 训练后模型：固定参数 θ_T 后得到的函数 N_{θ_T} 。

很多“神经网络是某种范畴对象”的说法，若不说明指的是这三者中的哪一个，就会含混。

2 监督学习的统一符号

定义 2.1 (数据集). 监督学习数据集记为

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathcal{X}, \quad y_i \in \mathcal{Y}.$$

这里 $\mathcal{X}$ 是输入空间, $\mathcal{Y}$ 是标签或输出空间。

典型例子如下:

任务	输入 x	输出 y
图像分类	图像张量 $x \in \mathbb{R}^{H \times W \times C}$	类别 $y \in \{1, \dots, K\}$
回归	特征向量 $x \in \mathbb{R}^d$	实数或向量 $y \in \mathbb{R}^m$
语言模型	token 序列 $x = (t_1, \dots, t_m)$	下一个 token t_{m+1}
序列到序列	输入序列 x	输出序列 y

定义 2.2 (损失函数). 模型 $N_\theta: \mathcal{X} \rightarrow \mathcal{Y}'$ 的损失函数通常写作

$$\ell(N_\theta(x), y) \in \mathbb{R}_{\geq 0}.$$

经验风险是训练集上的平均损失:

$$\mathcal{L}_{\mathcal{D}}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(N_\theta(x_i), y_i).$$

训练的基本目标是

$$\theta^* \approx \arg \min_{\theta \in \Theta} \mathcal{L}_{\mathcal{D}}(\theta).$$

注意 $\mathcal{Y}'$ 不一定等于 $\mathcal{Y}$ 。分类任务中, 标签 y 是类别, 而模型输出通常是 logits:

$$z = N_\theta(x) \in \mathbb{R}^K.$$

经过 softmax 得到概率分布

$$p_k = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}.$$

若真实类别为 y , 交叉熵损失为

$$\ell(z, y) = -\log p_y.$$

3 全连接神经网络

最基本的神经网络是若干仿射映射和非线性函数的复合。令

$$h_0 = x \in \mathbb{R}^{d_0}.$$

第 l 层定义为

$$a_l = W_l h_{l-1} + b_l, \quad h_l = \sigma_l(a_l),$$

其中

$$W_l \in \mathbb{R}^{d_l \times d_{l-1}}, \quad b_l \in \mathbb{R}^{d_l}.$$

参数为

$$\theta = (W_1, b_1, \dots, W_L, b_L).$$

输出为

$$N_\theta(x) = h_L.$$

说明 3.1. 从范畴论眼光看，层复合

$$N_\theta = f_{L, \theta_L} \circ \dots \circ f_{1, \theta_1}$$

像是态射复合。但机器学习还关心每个 f_{l, θ_l} 的参数如何变化、损失如何沿复合求导、以及哪些参数共享或受约束。这些信息不在普通态射复合记号中自动出现。

常用非线性包括

$$\text{ReLU}(u) = \max(u, 0), \quad \tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}.$$

非线性很重要。若没有非线性，多个线性层的复合仍是一个线性层：

$$W_L \cdots W_1 x.$$

于是深度不会增加表达能力。

4 训练与反向传播

训练通常使用随机梯度下降及其变体。给定 mini-batch

$$B \subseteq \{1, \dots, n\},$$

mini-batch 损失为

$$\mathcal{L}_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \ell(N_\theta(x_i), y_i).$$

最简单的梯度下降更新是

$$\theta_{t+1} = \theta_t - \eta_t \nabla_\theta \mathcal{L}_B(\theta_t),$$

其中 $\eta_t > 0$ 是学习率。

4.1 反向传播的链式法则

反向传播不是神秘算法，而是高效组织链式法则。设单个样本的损失为

$$\mathcal{L} = \ell(h_L, y).$$

定义误差信号

$$\delta_l = \frac{\partial \mathcal{L}}{\partial a_l} \in \mathbb{R}^{d_l}.$$

最后一层由损失函数给出 δ_L 。对中间层有递推

$$\delta_l = (W_{l+1}^\top \delta_{l+1}) \odot \sigma'_l(a_l),$$

其中 $\odot$ 是逐坐标乘法。于是参数梯度为

$$\frac{\partial \mathcal{L}}{\partial W_l} = \delta_l h_{l-1}^\top, \quad \frac{\partial \mathcal{L}}{\partial b_l} = \delta_l.$$

原则 4.1 (反向传播的范畴论读法)。若前向传播是复合

$$f_L \circ \cdots \circ f_1,$$

则反向传播沿相反方向传播导数信息。它不是把原函数反过来，而是把 *cotangent/gradients* 信息通过每一层的导数转置向后传递。

5 卷积网络：局部性和权重共享

卷积神经网络 CNN 主要用于图像和局部网格数据。输入可写成

$$x \in \mathbb{R}^{H \times W \times C_{\text{in}}}.$$

卷积核为

$$K \in \mathbb{R}^{r \times s \times C_{\text{in}} \times C_{\text{out}}}.$$

输出特征图 $y \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$ 的坐标为

$$y_{u,v,c} = \sum_{\alpha=1}^r \sum_{\beta=1}^s \sum_{c'=1}^{C_{\text{in}}} K_{\alpha,\beta,c',c} x_{u+\alpha,v+\beta,c'}.$$

这里有两个关键结构。

第一是局部性： $y_{u,v,c}$ 只依赖输入中靠近 (u, v) 的一个小窗口。第二是权重共享：同一个卷积核 K 在所有位置使用。这使 CNN 比全连接网络更适合表达平移相关的图像结构。

如果要用层或 sheaf 语言看 CNN，图像 patch 可以作为局部上下文；patch 之间的重叠给出相容性问题；卷积核共享则是一种对平移结构的约束。

6 图神经网络：邻域聚合

图神经网络 GNN 处理图

$$G = (V, E).$$

每个节点 $v \in V$ 有表示

$$h_v^{(0)} \in \mathbb{R}^{d_0}.$$

第 l 层通常由消息传递给出：

$$m_v^{(l)} = \text{AGG}^{(l)}\{\phi^{(l)}(h_v^{(l)}, h_u^{(l)}, e_{uv}) \mid u \in \mathcal{N}(v)\},$$

$$h_v^{(l+1)} = \psi^{(l)}(h_v^{(l)}, m_v^{(l)}).$$

这里 $\mathcal{N}(v)$ 是 v 的邻居集合，AGG 通常是求和、平均或最大值。

GNN 的关键是置换等变性。若重新编号节点，输出表示也应按同样方式重新编号，而不应依赖任意编号。这一点和范畴论中的自然性、等变性很接近，但具体证明依赖聚合函数是否对邻居顺序不敏感。

7 Transformer 的统一符号

Transformer 是现代 LLM 的核心。设词表为

$$\mathcal{V} = \{1, \dots, V\}.$$

一个长度为 T 的 token 序列写作

$$t_{1:T} = (t_1, \dots, t_T), \quad t_i \in \mathcal{V}.$$

嵌入矩阵为

$$E \in \mathbb{R}^{V \times d}.$$

token t_i 的向量表示是

$$x_i = E_{t_i} + p_i \in \mathbb{R}^d,$$

其中 p_i 是位置编码。把所有位置堆叠成矩阵

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_T^\top \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

7.1 Scaled dot-product attention

给定

$$X \in \mathbb{R}^{T \times d},$$

定义 query、key、value:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

其中

$$W_Q, W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v}.$$

attention 输出为

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

矩阵

$$\frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}$$

给出每个位置对每个位置的相似度。softmax 按行计算，因此第 i 行是位置 i 对所有位置 j 的权重。

在自回归语言模型中，mask M 用来禁止看未来 token:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

这保证预测第 $i+1$ 个 token 时只能使用 $t_1, \dots, t_i$ 。

7.2 Multi-head attention

多头注意力把 H 个 attention head 并行计算:

$$\text{head}_r = \text{Attn}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}), \quad r = 1, \dots, H.$$

然后拼接并线性变换:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O.$$

多头机制允许不同 head 学习不同关系，例如局部语法、长程依赖、实体指代或格式模式。这里的“关系”不是人工指定的逻辑关系，而是由训练数据和损失函数诱导出来的统计结构。

7.3 Transformer block

一个常见 decoder-only Transformer block 可写为

$$Y = X + \text{MHA}(\text{LN}(X)),$$

$$Z = Y + \text{FFN}(\text{LN}(Y)).$$

其中 FFN 是逐位置的两层 MLP:

$$\text{FFN}(u) = W_2 \sigma(W_1 u + b_1) + b_2.$$

注意 attention 在 token 位置之间混合信息，而 FFN 对每个位置分别作用。

8 语言模型目标

语言模型学习条件概率

$$p_\theta(t_{1:T}) = \prod_{i=1}^T p_\theta(t_i \mid t_{<i}).$$

训练目标通常是最大化训练文本的似然，等价于最小化负对数似然:

$$\mathcal{L}(\theta) = - \sum_{i=1}^T \log p_\theta(t_i \mid t_{<i}).$$

模型在每个位置输出 logits

$$z_i \in \mathbb{R}^{|\mathcal{V}|}.$$

概率为

$$p_\theta(t_{i+1} = v \mid t_{\leq i}) = \text{softmax}(z_i)_v.$$

说明 8.1. 从数学上看，LLM 不是直接输出“意义”，而是输出下一个 *token* 的条件分布。意义、推理、工具使用和知识检索是从大规模训练、架构约束、上下文输入和解码策略中涌现或被工程化出来的行为。

9 推理、采样与 KV cache

训练后，给定 prompt

$$c = (t_1, \dots, t_m),$$

模型逐步生成:

$$t_{m+1} \sim p_\theta(\cdot \mid t_{\leq m}),$$

$$t_{m+2} \sim p_\theta(\cdot \mid t_{\leq m+1}),$$

依此类推。

常见解码方式包括:

$$t_{i+1} = \arg \max_{v \in \mathcal{V}} p_\theta(v \mid t_{\leq i})$$

的 greedy decoding，以及按概率采样的 stochastic decoding。temperature $\tau > 0$ 调整分布尖锐程度:

$$p_\tau(v) = \text{softmax}(z/\tau)_v.$$

τ 越小，输出越确定； τ 越大，输出越随机。

KV cache 是推理加速结构。由于每一步都会重复使用过去 token 的 key 和 value，模型把已经计算的

$$K_{\leq i}, \quad V_{\leq i}$$

缓存起来。生成第 $i+1$ 个 token 时，只需为新 token 计算新的 q, k, v ，再与缓存交互。这不改变模型定义，只改变计算效率。

10 RAG 与工具调用

检索增强生成 RAG 把外部文档引入上下文。给定查询 q ，检索器从文档库

$$\mathcal{R} = \{d_1, \dots, d_M\}$$

取出相关文档

$$r_1, \dots, r_k.$$

然后模型条件化于扩展上下文：

$$p_\theta(t_{i+1} \mid q, r_1, \dots, r_k, t_{\leq i}).$$

这不是改变 Transformer 的基本公式，而是改变输入上下文和信息来源。

工具调用可抽象为模型在某一步输出一个结构化 action

$$a_i \in \mathcal{A},$$

外部环境返回观察

$$o_i \in \mathcal{O}.$$

之后模型继续在扩展历史

$$(t_{\leq i}, a_i, o_i)$$

上生成。若要用范畴论语言描述工具调用，态射不应只表示文本变换，还要表示模型、工具、环境之间的信息流。

11 这些 AI 对象怎样和范畴/topos 问题对接

现在可以更精确地说 AI 侧有哪些候选结构。

AI 对象	可能的范畴论/topos 对接点
模型族 $\{N_\theta\}_{\theta \in \Theta}$	参数化态射、函数对象、模型空间
训练轨迹 $\theta_0, \dots, \theta_T$	优化动力系统，不等同于固定态射

层 f_l, θ_l	复合结构、局部模块、可替换组件
CNN patch	局部上下文、覆盖、限制映射
GNN 邻域	局部邻域聚合、置换等变性
attention 矩阵	token 间上下文依赖，不是简单局部限制
LLM prompt	条件上下文，可作为某类 context object
RAG 文档	外部证据片段，可能形成覆盖或索引数据
工具调用	模型与环境的交互态射或过程

原则 11.1. 若要把一个 *AI* 系统放入 *topos* 框架，不能只说“存在上下文”。必须说明上下文对象是什么、态射是什么、覆盖是什么、局部数据是什么、相容性是什么、粘合后得到什么。

12 思维链的谨慎解释

Chain-of-thought 可以看成中间文本轨迹

$$c = (s_1, \dots, s_m),$$

其中每个 s_i 是一个中间推理片段。但这些片段不一定彼此一致。模型可能先提出错误假设，再修正；也可能并列比较多个方向。

因此，用 sheaf 的“局部相容则全局粘合”直接解释 chain-of-thought 会过于简单。更合理的做法是把思维链看成一种动态过程：

$$\text{state}_0 \rightarrow \text{state}_1 \rightarrow \dots \rightarrow \text{state}_m,$$

其中态射可能表示扩展、反驳、修正、压缩或验证。若要建立范畴模型，应先决定这些态射的类型，而不是预设它们都是限制映射。

13 最小词汇表

AI 术语	本讲义符号
输入空间	$\mathcal{X}$
输出/标签空间	$\mathcal{Y}$
参数空间	Θ
模型	$N_\theta : \mathcal{X} \rightarrow \mathcal{Y}'$
数据集	$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$
损失	$\mathcal{L}_{\mathcal{D}}(\theta)$
隐藏状态	h_l 或 $X \in \mathbb{R}^{T \times d}$
logits	$z \in \mathbb{R}^K$ 或 $z_i \in \mathbb{R}^{ \mathcal{V} }$
token 序列	$t_{1:T}$
语言模型概率	$p_\theta(t_i t_{<i})$
attention	$\text{Attn}(Q, K, V)$

14 结语

范畴论读者理解 AI 算法时，最重要的是不要把所有东西都压成一个函数。固定参数后的网络确实是函数；但机器学习真正关心的是参数化函数族、训练目标、优化路径、局部模块、上下文依赖、概率输出和交互过程。只有这些对象被清楚区分之后，讨论预层、层、stack 或 topos 才有可靠的落点。